



Chapter 10

Trees

MAD101

Ly Anh Duong

duongla3@fe.edu.vn





Table of Contents

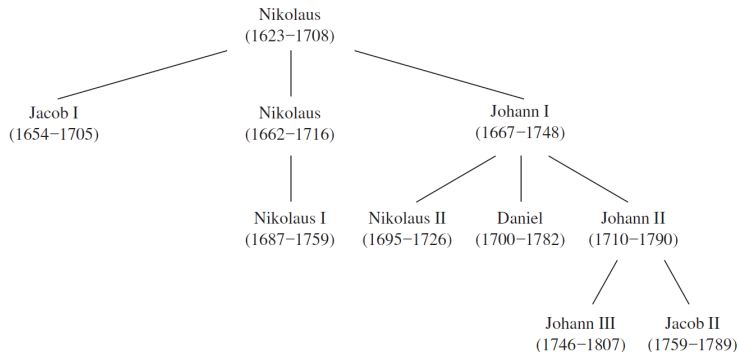
1 Introduction to Trees

- ▶ Introduction to Trees
- ▶ Binary Search Tree
- ▶ Prefix code
- ▶ Decision tree
- ▶ Tree Traversal
- ▶ Infix, Prefix, and Postfix Notation
- ▶ Spanning Trees
- ▶ Min.Span.Trees
- ▶ Quiz

The Bernoulli family of mathematicians

1 Introduction to Trees

Family trees use vertices to represent the members of a family and edges to represent parent–child relationships. The family tree of the male members of the Bernoulli family of Swiss mathematicians is shown in Figure.



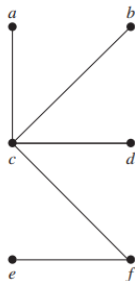
Definition

1 Introduction to Trees

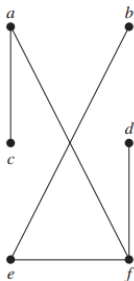
Definition 1.1

A **tree** is a **connected undirected graph** with **no simple circuits**.

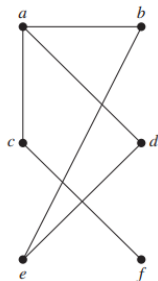
Example 1.1. G_1 , G_2 are trees and G_3 , G_4 are not.



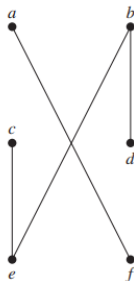
G_1



G_2



G_3



G_4

Definition - Rooted Trees

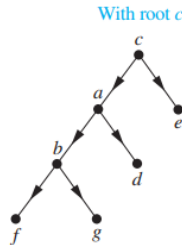
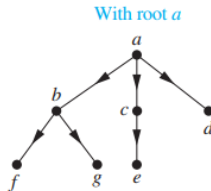
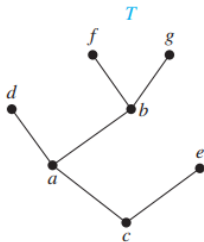
1 Introduction to Trees

Theorem 1.1

An **undirected** graph is a tree if and only if there is a unique simple path between any two of its vertices.

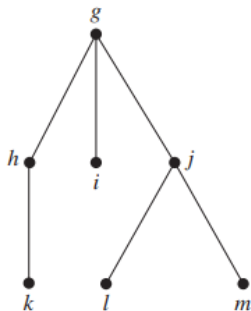
Definition 1.2

A **rooted tree** is a tree in which one vertex has been designated as the root and every edge is directed away from the root.



Some common concepts

1 Introduction to Trees



- g is **root** of tree
- j is **parent** of m and l
- m or l is a **child** of j
- m, l are **siblings**
- g is **ancestor** of m and m is **descendant** of g
- i, k, l, m are **leaves**: nodes without children.
- g, h, j are **internal nodes/vertices**: nodes have children.

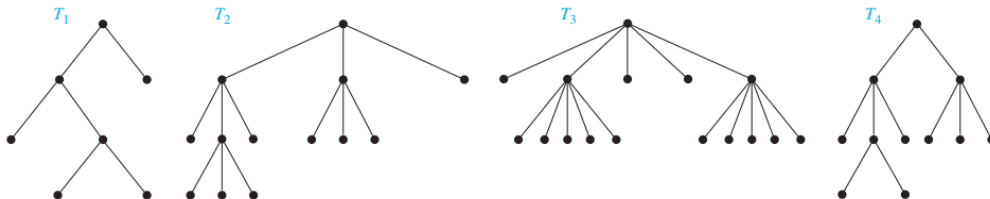
m-ary tree

1 Introduction to Trees

Definition 1.3

- A rooted tree is called an **m-ary** tree if every internal vertex has no more than m children.
- The tree is called a **full m-ary** tree if every internal vertex has exactly m children.
- An m -ary tree with $m = 2$ is called a **binary tree**.

Example 1.2.



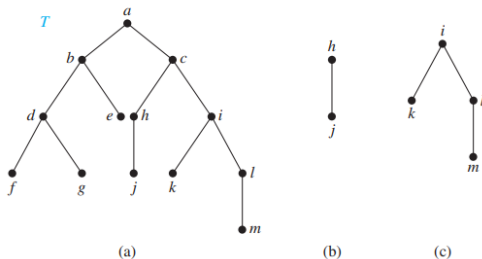
Ordered rooted tree

1 Introduction to Trees

Definition 1.4

An **ordered rooted tree** is a rooted tree where the children of each internal vertex are ordered (which is usually represented as in the graph).

Example 1.3. f is the left child of d and g is the right child of d ; (b): the left subtree of c and (c): the right subtree of c .



Properties of trees

1 Introduction to Trees

Theorem 1.2

1. A tree with n vertices has $n - 1$ edges.
2. For a m -ary tree

$$n = i + l$$

3. For a full m -ary tree

$$n = mi + 1$$

- n are number of nodes,
- i are number of internal nodes,
- l are number of leaves.

Example 1.4.

1. How many leaves are there in a full 5-ary tree with 56 nodes.
2. How many vertices are there in a full ternary (3-ary) tree with 27 leaves.

Properties of trees

1 Introduction to Trees

Theorem 1.3

1. A tree with n vertices has $n - 1$ edges.
2. For a m -ary tree

$$n = i + l$$

3. For a full m -ary tree

$$n = mi + 1$$

- n are number of nodes,
- i are number of internal nodes,
- l are number of leaves.

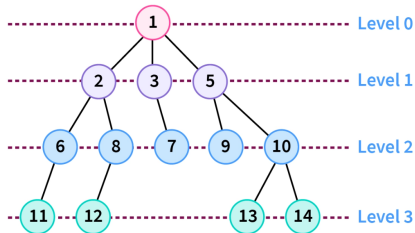
Example 1.5.

1. How many leaves are there in a full 5-ary tree with 56 nodes.
2. How many vertices are there in a full ternary (3-ary) tree with 27 leaves.

Solution. 1. $l = 45$, 2. $n = 40$

BALANCED m -ARY TREES

1 Introduction to Trees



Definition 1.5

- The **level** of a vertex v in a rooted tree is the length of the unique path from the root to this vertex.
- The **level of the root** is defined to be **zero**.
- The **height h** of a rooted tree is the maximum of the levels of vertices.
- A rooted m -ary tree of height h is **balanced** if **all leaves are at levels h or $h - 1$** .

Theorem 1.4

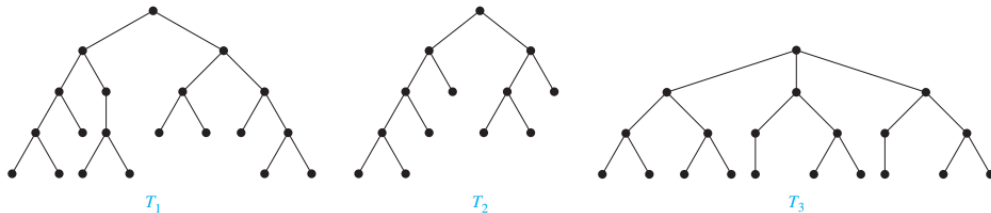
There are at **most m^h leaves** in an m -ary tree of height h .

Corollary 1.0

If an m -ary tree of height h has l leaves, then $h \geq \lceil \log_m l \rceil$.
If the m -ary tree is full and balanced, then $h = \lceil \log_m l \rceil$.

BALANCED m-ARY TREES - Example

1 Introduction to Trees



1. There are 10 nodes leaves in T_1 and their level are 3 & 4 ($h=4$) $\implies$ balanced
2. There are 7 nodes leaves in T_2 and their level are 2, 3 & 4 ($h=4$) $\implies$ NOT balanced
3. There are 10 nodes leaves in T_3 and their level are 3 ($h=3$) $\implies$ balanced



Table of Contents

2 Binary Search Tree

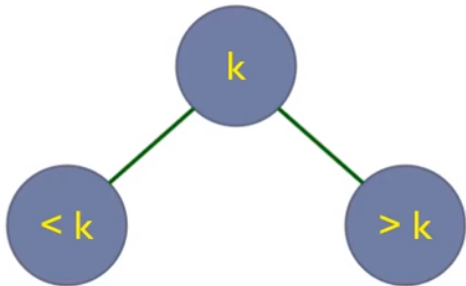
- ▶ Introduction to Trees
- ▶ Binary Search Tree
- ▶ Prefix code
- ▶ Decision tree
- ▶ Tree Traversal
- ▶ Infix, Prefix, and Postfix Notation
- ▶ Spanning Trees
- ▶ Min.Span.Trees
- ▶ Quiz

Definition

2 Binary Search Tree

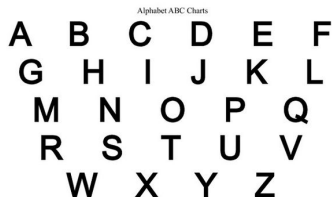
Definition 2.1

- A binary search tree where each vertex is labeled with a key.
- The key of a vertex is both **larger than the keys** of all vertices in its **left subtree** and **smaller than the keys** of all vertices in its **right subtree**.



Alphabetical order

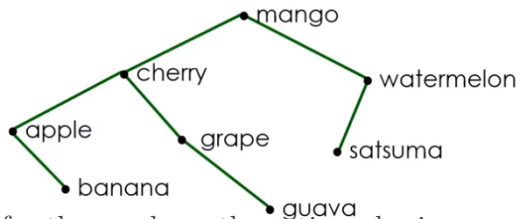
2 Binary Search Tree



$a < b < c \dots$; $ab < ac < ae \dots$;
 $abc < abd < abe \dots$

Example 2.2. Form a binary search tree for the words mathematics, physics, geography, zoology, meteorology, geology, psychology, and chemistry (using alphabetical order).

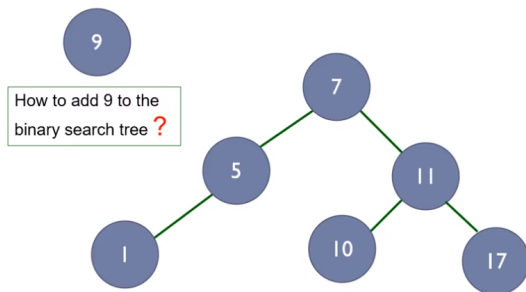
Example 2.1. Binary search tree for words: mango, cherry, grape, apple, guava, watermelon, satsuma, banana.



Example

2 Binary Search Tree

Example 2.3.



Example 2.4. Construct the Binary Search Trees for 8, 3, 11, 15, 2, 4, 14, 6. How many comparisons are required to locate number 6 in the search tree?

Algorithm 1

2 Binary Search Tree

ALGORITHM 1 Locating an Item in or Adding an Item to a Binary Search Tree.

```

procedure insertion( $T$ : binary search tree,
 $x$ : item)
 $v := \text{root of } T$ 
{a vertex not present in  $T$  has the value null }
while  $v \neq \text{null}$  and  $\text{label}(v) \neq x$ 
    if  $x < \text{label}(v)$  then
        if left child of  $v \neq \text{null}$  then  $v := \text{left child of } v$ 
        else add new vertex as a left child of  $v$  and set  $v := \text{null}$ 
    else
        if right child of  $v \neq \text{null}$  then  $v := \text{right child of } v$ 
        else add new vertex as a right child of  $v$  and set  $v := \text{null}$ 
if root of  $T = \text{null}$  then add a vertex  $v$  to the tree and label it with  $x$ 
else if  $v$  is null or  $\text{label}(v) \neq x$  then label new vertex with  $x$  and let  $v$  be this new vertex
return  $v$  {  $v = \text{location of } x$  }

```

Remark 2.1. The algorithm returns the location of x or adds a new vertex with label x into binary search tree. Complexity $O(\log n)$.



Table of Contents

3 Prefix code

- ▶ Introduction to Trees
- ▶ Binary Search Tree
- ▶ Prefix code
- ▶ Decision tree
- ▶ Tree Traversal
- ▶ Infix, Prefix, and Postfix Notation
- ▶ Spanning Trees
- ▶ Min.Span.Trees
- ▶ Quiz

Introduction

3 Prefix code

- Standard ASCII (*American Standard Code for Information Interchange*) character encoding: use 8 bits (1 byte) to store each character.
- For example, using the standard ASCII encoding, the 12- characters string "no one knows" requires $12 * 8 = 96$ bits total.

char	ASCII	bit pattern
n	110	01101110
o	111	01101111
e	101	01100101
k	107	01101011
w	119	01100111
s	115	01110011
space	32	00100000



Introduction

3 Prefix code

- A better code scheme

char	number	bit pattern
n	0	000
o	1	001
e	2	010
k	3	011
w	4	100
s	5	101
space	6	110

$12 * 3 = 36$ bits total.

- A variable-length encoding

char	bit pattern
n	01
o	11
e	000
k	001
w	1000
s	1001
space	101

Use 32 bits total.



Introduction

3 Prefix code

Consider the codes

1. Use the following code

$a : 10$	$e : 01$	$t : 0110$	$n : 1$
----------	----------	------------	---------

Decode the message 01101

01101 mean "ean"

01101 mean "tn"

Hence, it is not a prefix code.

2. Use the following code

$a : 10$	$e : 01$	$t : 001$	$n : 11$
----------	----------	-----------	----------

Decode the message 1101001

1101001 mean "net" only.

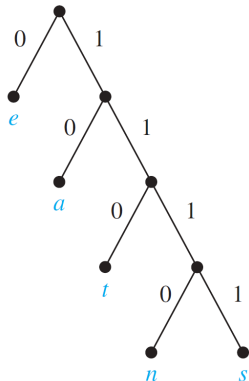
It is **Prefix code**

Definition 3.1

Prefix code is a type of codes which encodes the letters so that the bit string for a letter never occurs as the first part of the bit string for another letter.

A binary tree with a prefix code

3 Prefix code



$s : 1111$

$a : 10$

$n : 1110$

$e : 0$

“sane” will be store as 11111011100 $\rightarrow$ 11 bits.

Hence, compression factor: $32/11 \sim 3$.

Is it best?

How to construct a prefix code that is using the fewest bits?

Huffman coding algorithm

3 Prefix code

- Find frequencies (probabilities) of each character in a text.
- Constructing a binary tree representing prefix codes of character.

Remark 3.1.

- Sort by frequencies (probabilities) from small to large after each step.
- The **larger weight** sub-tree is on the **left** of the binary tree.

ALGORITHM 2 Huffman Coding.

procedure *Huffman*(C : symbols a_i with frequencies w_i , $i = 1, \dots, n$)

$F :=$ forest of n rooted trees, each consisting of the single vertex a_i and assigned weight w_i

while F is not a tree

Replace the rooted trees T and T' of least weights from F with $w(T) \geq w(T')$ with a tree having a new root that has T as its left subtree and T' as its right subtree. Label the new edge to T with 0 and the new edge to T' with 1.

Assign $w(T) + w(T')$ as the weight of the new tree.

{the Huffman coding for the symbol a_i is the concatenation of the labels of the edges in the unique path from the root to the vertex a_i }

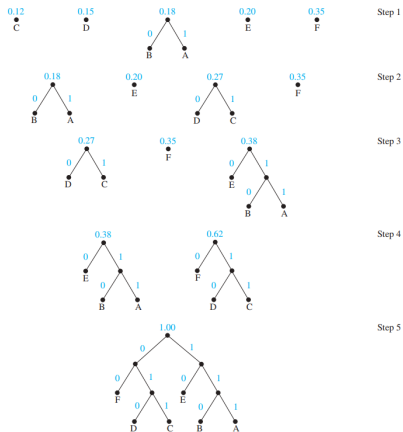
Huffman coding algorithm - Example

3 Prefix code

Example 3.1. Use Huffman coding to encode the following symbols with the frequencies listed: A: 0.08, B: 0.10, C: 0.12, D: 0.15, E: 0.20, F: 0.35. What is the average number of bits used to encode a character?

Solution. The average number of bits used to encode a symbol using this encoding is

$$3 \times 0.08 + 3 \times 0.10 + 3 \times 0.12 + 3 \times 0.15 + 2 \times 0.20 + 2 \times 0.35 = 2.45$$





Huffman coding algorithm - Example

3 Prefix code

Example 3.2. Use Huffman coding algorithm to encode the word “google”.

Example 3.3. Use Huffman coding algorithm to encode the word “maximum”.



Table of Contents

4 Decision tree

- ▶ Introduction to Trees
- ▶ Binary Search Tree
- ▶ Prefix code
- ▶ **Decision tree**
- ▶ Tree Traversal
- ▶ Infix, Prefix, and Postfix Notation
- ▶ Spanning Trees
- ▶ Min.Span.Trees
- ▶ Quiz

A counterfeit coin problem

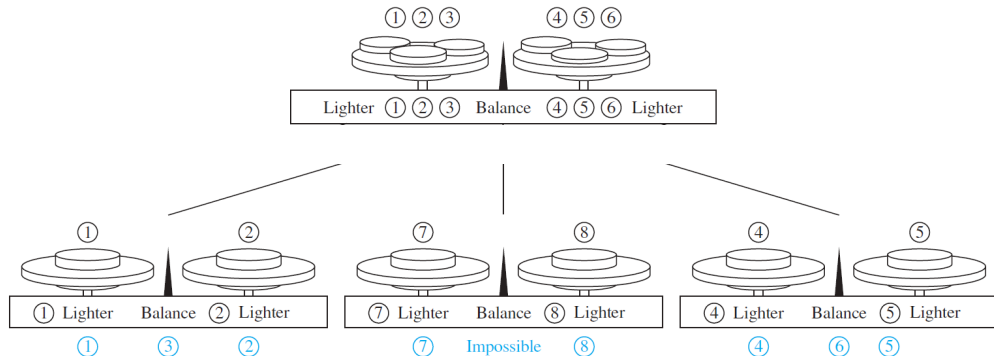
4 Decision tree



Suppose there are seven coins, all with the same weight, and a counterfeit coin that weighs less than the others. **How many weighings** are necessary using a balance scale to determine which of the eight coins is the counterfeit one?

Solution

4 Decision tree



Thus, $n = 8$, we need $2 = \lceil \log_3 8 \rceil = 2$ weighings. How about $n = 3, n = 9, n = 27$?

→ In general, we need $\lceil \log_3 n \rceil$ weighings to find the counterfeit coin.



Definition

4 Decision tree

Definition 4.1

A rooted tree in which each internal vertex corresponds to a decision, with a subtree at these vertices for each possible outcome of the decision, is called a **decision tree**.

Theorem 4.1

A sorting algorithm based on binary comparisons requires at least $\lceil \log n! \rceil$ comparisons.

Corollary 4.0

The number of comparisons used by a sorting algorithm to sort n elements based on binary comparisons is $\Omega(n \log n)$.

Theorem 4.2

The average number of comparisons used by a sorting algorithm to sort n elements based on binary comparisons is $\Omega(n \log n)$.



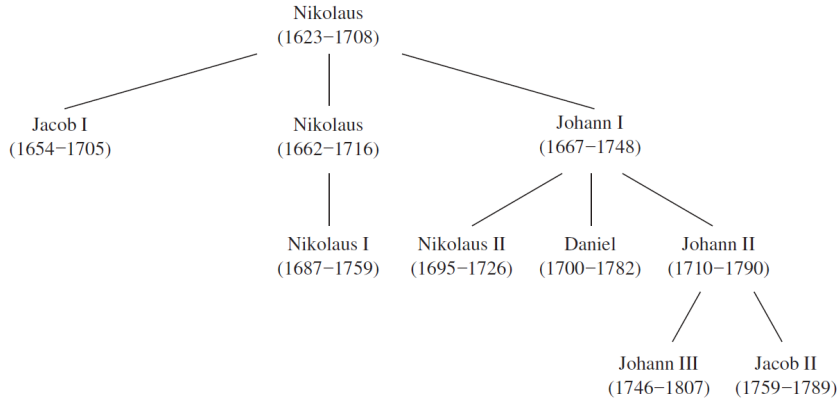
Table of Contents

5 Tree Traversal

- ▶ Introduction to Trees
- ▶ Binary Search Tree
- ▶ Prefix code
- ▶ Decision tree
- ▶ **Tree Traversal**
- ▶ Infix, Prefix, and Postfix Notation
- ▶ Spanning Trees
- ▶ Min.Span.Trees
- ▶ Quiz

Introduction

5 Tree Traversal



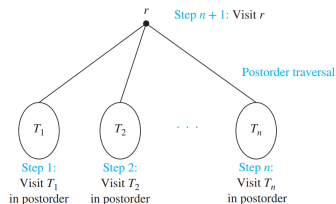
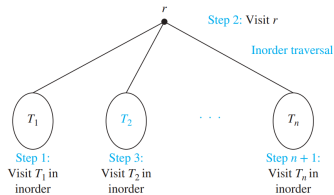
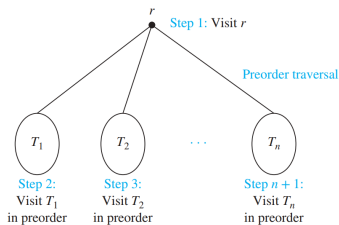
Traversal a tree: A way to visit all vertices of the rooted tree.

Traversal Algorithms

5 Tree Traversal

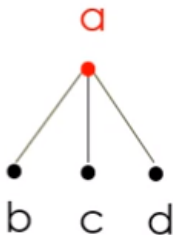
- At a time, a vertex is visited,
- Recursive algorithm,
- Bases on orders of tasks, traversals are classified into:
 1. **Pre**-order traversal. **N** L R
 2. **In**-order traversal. L **N** R
 3. **Post**-order traversal. L R **N**

where **N**: root node, **L**: left subtree, **R**: right subtree



Example

5 Tree Traversal



Pre-order

N L R

a b c d

In-order

L N R

b a c d

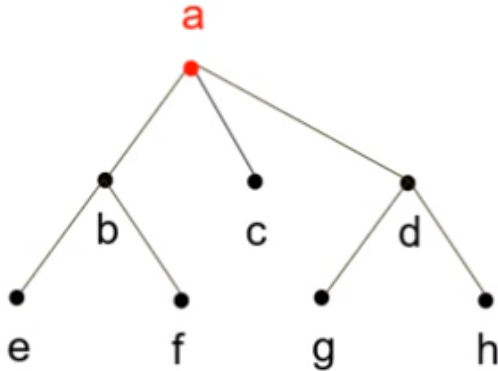
Post-order

L R N

b c d a

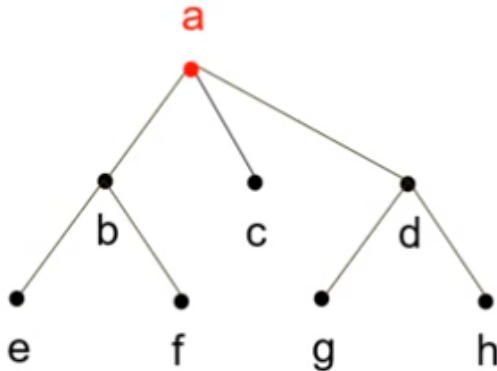
Pre-order traversal example

5 Tree Traversal



Pre-order traversal example

5 Tree Traversal



Pre-order traversal: a b e f c d g h



Pre-order traversal algorithm

5 Tree Traversal

ALGORITHM 1 Preorder Traversal.

procedure *preorder*(T : ordered rooted tree)

$r := \text{root of } T$

list r

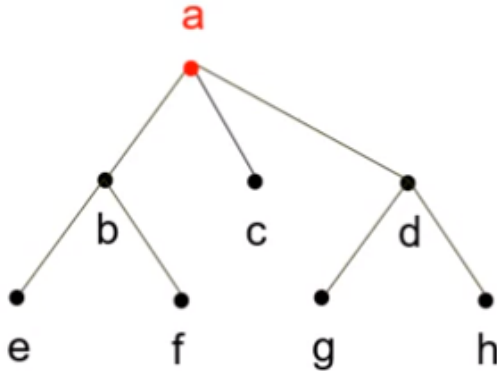
for each child c of r from left to right

$T(c) := \text{subtree with } c \text{ as its root}$

preorder($T(c)$)

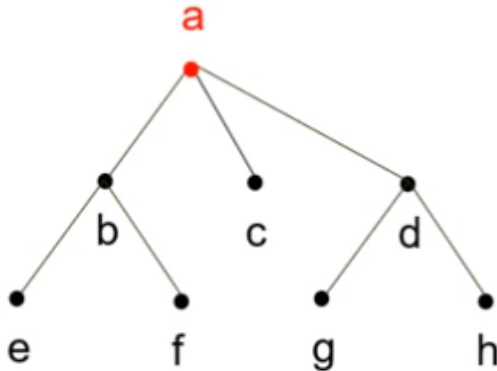
In-order traversal example

5 Tree Traversal



In-order traversal example

5 Tree Traversal



In-order traversal: e b f a c g d h

In-order traversal algorithm

5 Tree Traversal

ALGORITHM 2 Inorder Traversal.

procedure *inorder*(T : ordered rooted tree)

$r :=$ root of T

if r is a leaf **then** list r

else

$l :=$ first child of r from left to right

$T(l) :=$ subtree with l as its root

inorder($T(l)$)

 list r

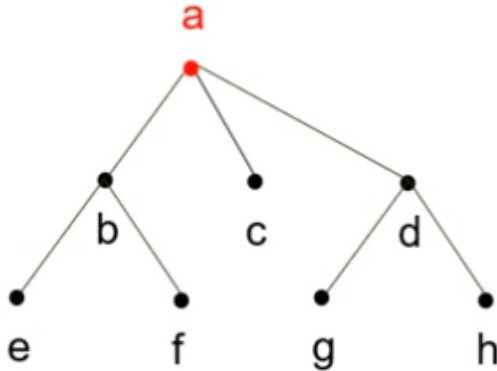
for each child c of r except for l from left to right

$T(c) :=$ subtree with c as its root

inorder($T(c)$)

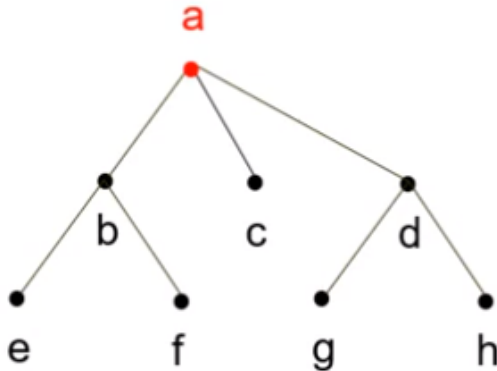
Post-order traversal example

5 Tree Traversal



Post-order traversal example

5 Tree Traversal



Post-order traversal: e f b c g h d a



Post-order traversal algorithm

5 Tree Traversal

ALGORITHM 3 Postorder Traversal.

procedure *postorder*(T : ordered rooted tree)

$r := \text{root of } T$

for each child c of r from left to right

$T(c) := \text{subtree with } c \text{ as its root}$

postorder($T(c)$)

list r



Exercise

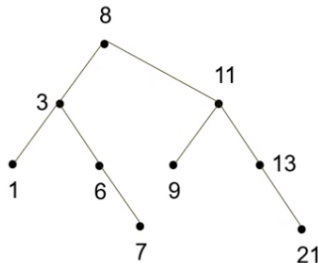
5 Tree Traversal

Construct a **binary search tree** for the numbers: 8, 11, 3, 6, 9, 1, 13, 7, 21. What are the order of numbers after applying:

1. preorder traversal
2. inorder traversal
3. postorder traversal

Solution

5 Tree Traversal



Binary search tree

1. Pre-order: 8 3 1 6 7 11 9 13 21
2. In-order: 1 3 6 7 8 9 11 13 21
3. Post-order: 1 7 6 3 9 21 13 11 8

Remark 5.1. In in-order, the sequences of numbers are sorted.



Table of Contents

6 Infix, Prefix, and Postfix Notation

- ▶ Introduction to Trees
- ▶ Binary Search Tree
- ▶ Prefix code
- ▶ Decision tree
- ▶ Tree Traversal
- ▶ Infix, Prefix, and Postfix Notation
- ▶ Spanning Trees
- ▶ Min.Span.Trees
- ▶ Quiz



Representing the expression

6 Infix, Prefix, and Postfix Notation

- $+$: addition
- $-$: subtraction
- $*$: multiplication
- $/$: division
- $\uparrow$: exponentiation

The expression can be represent by the binary trees with the rules:

- The internal vertices represent operations,
- The leaves represent the variables or numbers.



Example

6 Infix, Prefix, and Postfix Notation

Example 6.1. What is the ordered rooted tree that represents the expression

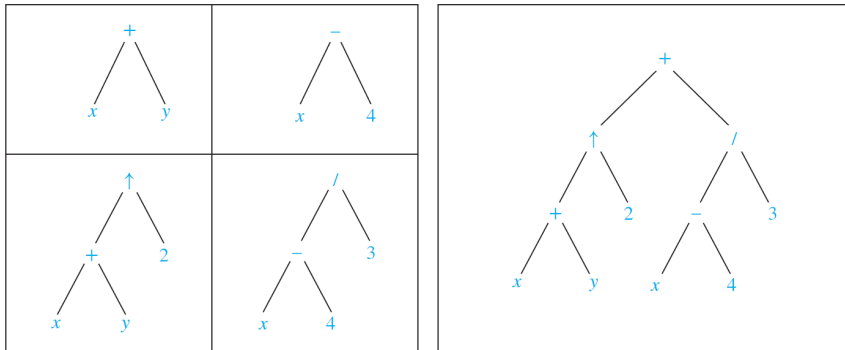
$$((x + y) \uparrow 2) + ((x - 4)/3)?$$

Example

6 Infix, Prefix, and Postfix Notation

Example 6.2. What is the ordered rooted tree that represents the expression

$$((x + y) \uparrow 2) + ((x - 4)/3)?$$

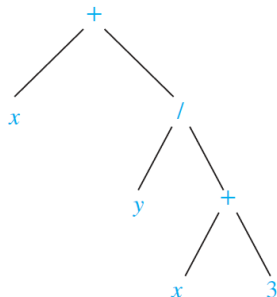
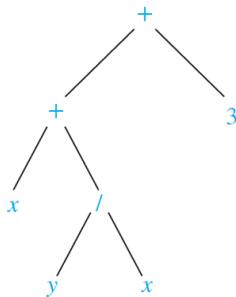
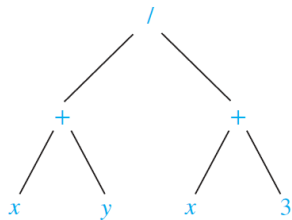


Example

6 Infix, Prefix, and Postfix Notation

What is the ordered rooted tree that represents the expression

$(x + y)/(x + 3)$, $(x + (y/x)) + 3$ and $x + (y/(x + 3))$?





Definition

6 Infix, Prefix, and Postfix Notation

Theorem 6.1

1. We obtain **prefix form** of an expression when we traverse its rooted tree in preorder $+ \textcolor{red}{x} \textcolor{red}{y}$
2. We obtain **infix form** of an expression when we traverse its rooted tree in inorder $\textcolor{red}{x} + \textcolor{red}{y}$
3. We obtain **postfix form** of an expression when we traverse its rooted tree in postorder $\textcolor{red}{x} \textcolor{red}{y} +$

Remark 6.1. Prefix form is called **Polish notation** and Postfix form is called **reverse Polish notation**.

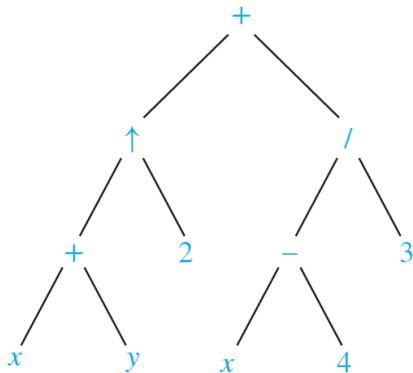
Remark 6.2. How to find prefix and postfix form from infix form?

1. Draw expression tree.
2. Using Preorder traverse $\rightarrow$ Prefix form. Using Postorder traverse $\rightarrow$ Postfix form.

Example

6 Infix, Prefix, and Postfix Notation

Example 6.3. What is the prefix (infix, postfix) form of the following tree?



- Infix form: $((x + y) \uparrow 2) + ((x - 4)/3)$
- Prefix form: $+ \uparrow +xy2 / - x43$
- Postfix form: $xy + 2 \uparrow x4 - 3 / +$



Evaluate a Prefix and Postfix form

6 Infix, Prefix, and Postfix Notation

1. Postfix form, work from left to right.
2. Prefix form, work from right to left.

Example 6.4.

1. $+ - * 235 / \uparrow 234$
2. $723 * -4 \uparrow 93 / +$

Solution

6 Infix, Prefix, and Postfix Notation

$$+ \quad - \quad * \quad 2 \quad 3 \quad 5 \quad / \quad \underbrace{\uparrow \quad 2 \quad 3}_{} \quad 4$$

$$2 \uparrow 3 = 8$$

$$+ \quad - \quad * \quad 2 \quad 3 \quad 5 \quad / \quad \underbrace{8 \quad 4}_{} \quad$$

$$8 / 4 = 2$$

$$+ \quad - \quad \underbrace{* \quad 2 \quad 3}_{} \quad 5 \quad 2$$

$$2 * 3 = 6$$

$$+ \quad \underbrace{- \quad 6 \quad 5}_{} \quad 2$$

$$6 - 5 = 1$$

$$\underbrace{+ \quad 1 \quad 2}_{} \quad$$

$$1 + 2 = 3$$

Value of expression: 3

$$7 \quad \underbrace{2 \quad 3 \quad *}_{} \quad - \quad 4 \quad \uparrow \quad 9 \quad 3 \quad / \quad +$$

$$2 * 3 = 6$$

$$\underbrace{7 \quad 6 \quad -}_{} \quad 4 \quad \uparrow \quad 9 \quad 3 \quad / \quad +$$

$$7 - 6 = 1$$

$$\underbrace{1 \quad 4 \quad \uparrow}_{} \quad 9 \quad 3 \quad / \quad +$$

$$1^4 = 1$$

$$1 \quad \underbrace{9 \quad 3 \quad /}_{} \quad +$$

$$9 / 3 = 3$$

$$\underbrace{1 \quad 3 \quad +}_{} \quad$$

$$1 + 3 = 4$$

Value of expression: 4



Table of Contents

7 Spanning Trees

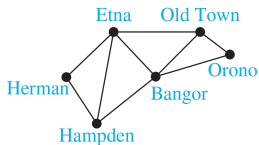
- ▶ Introduction to Trees
- ▶ Binary Search Tree
- ▶ Prefix code
- ▶ Decision tree
- ▶ Tree Traversal
- ▶ Infix, Prefix, and Postfix Notation
- ▶ **Spanning Trees**
- ▶ Min.Span.Trees
- ▶ Quiz

Introduction

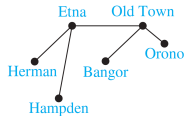
7 Spanning Trees

- The only way the roads can be kept open in the winter is by frequently plowing them,
- The highway department wants to plow the fewest roads so that there will always be cleared roads connecting any two towns,
- How can this be done?

The system of roads in Maine (USA)



At least five roads must be plowed to ensure that there is a path between any two towns



Spanning Trees

7 Spanning Trees

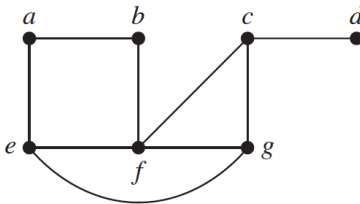
Definition 7.1

Let G be a simple graph. A **spanning tree** of G is a subgraph of G that is a tree containing every vertex of G .

Theorem 7.1

A simple graph is connected if and only if it has a spanning tree.

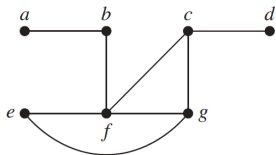
Example 7.1. Find a spanning tree of this simple graph:



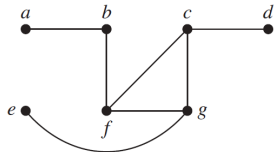
Solution

7 Spanning Trees

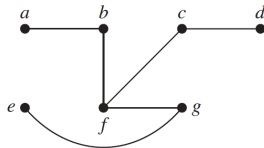
- Removed $\{a, e\}$



- Removed $\{e, f\}$



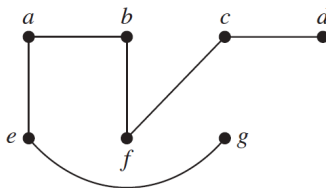
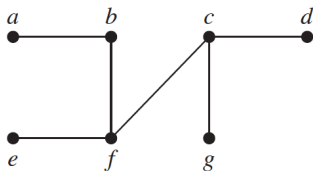
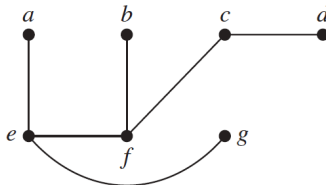
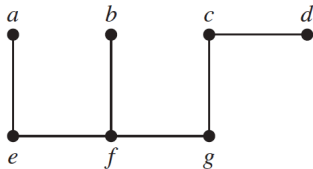
- Removed $\{c, g\}$



Solution

7 Spanning Trees

The spanning tree above is not unique, there are four different spanning trees





Depth-First Search (backtracking)

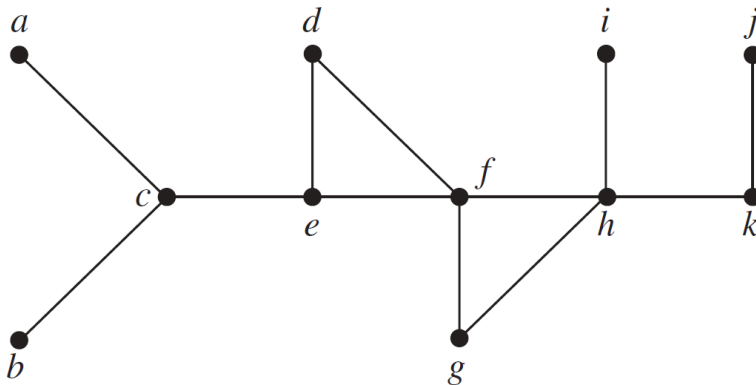
7 Spanning Trees

- Arbitrarily choose a vertex of the graph as the root. Form a path starting at this vertex by successively adding vertices and edges, where each new edge is incident with the last vertex in the path and a vertex not already in the path.
- Continue adding vertices and edges to this path as long as possible,
 1. If the path goes through all vertices of the graph, the tree consisting of this path is a spanning tree.
 2. If the path does not go through all vertices, move back to the next to last vertex in the path, forming new paths (adding new vertices and edges) that are as long as possible until no more edges can be added → spanning tree.

Example

7 Spanning Trees

Find a spanning tree of this graph using Depth-First Search:



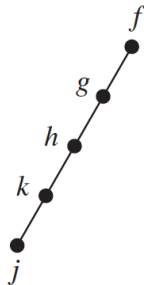
Solution

7 Spanning Trees

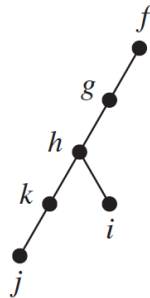
f



(a)



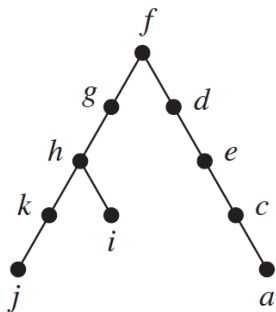
(b)



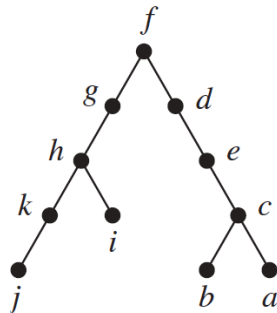
(c)

Solution

7 Spanning Trees



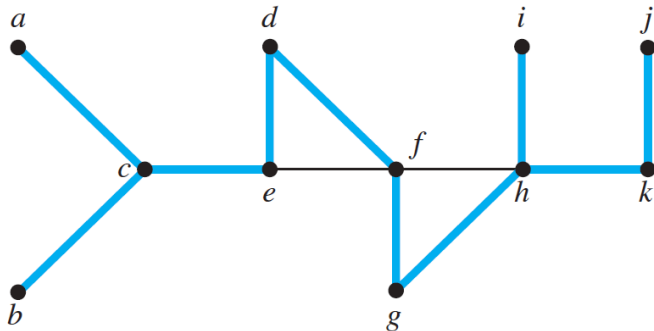
(d)



(e)

Tree edges and back edges

7 Spanning Trees

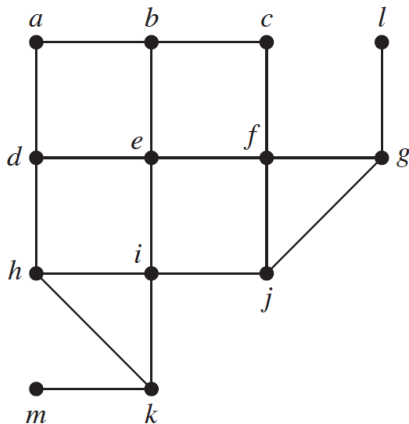


- Tree edges: highlight,
- Back edges: thinner black lines.

Example

7 Spanning Trees

Find a spanning tree of this graph using Depth-First Search:



Depth-First Search Algorithm

7 Spanning Trees

ALGORITHM 1 Depth-First Search.

procedure *DFS*(G : connected graph with vertices $v_1, v_2, \dots, v_n$)

$T :=$ tree consisting only of the vertex v_1

visit(v_1)

procedure *visit*(v : vertex of G)

for each vertex w adjacent to v and not yet in T

 add vertex w and edge $\{v, w\}$ to T

visit(w)



Breadth-First Search

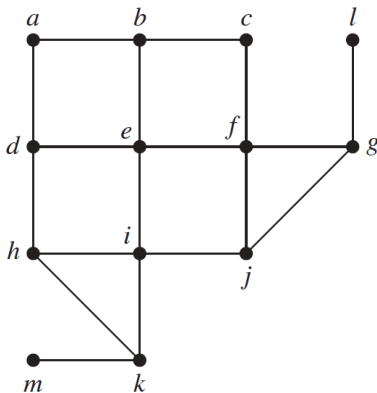
7 Spanning Trees

- Arbitrarily choose a root from the vertices of the graph, then add all edges incident to this vertex (level 1),
- For each vertex at level 1, add each edge incident to this vertex to the tree as long as it does not produce a simple circuit (level 2),
- The same procedure until all the vertices in the tree have been added.

Example

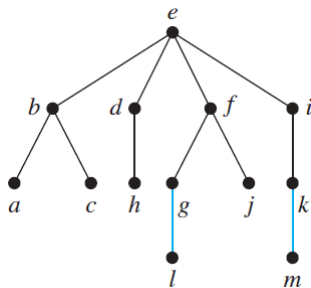
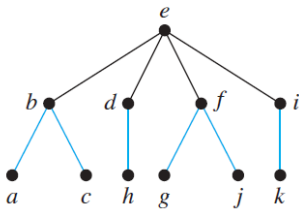
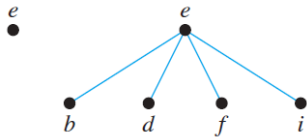
7 Spanning Trees

Find a spanning tree of this graph using Breadth-First Search:



Solution

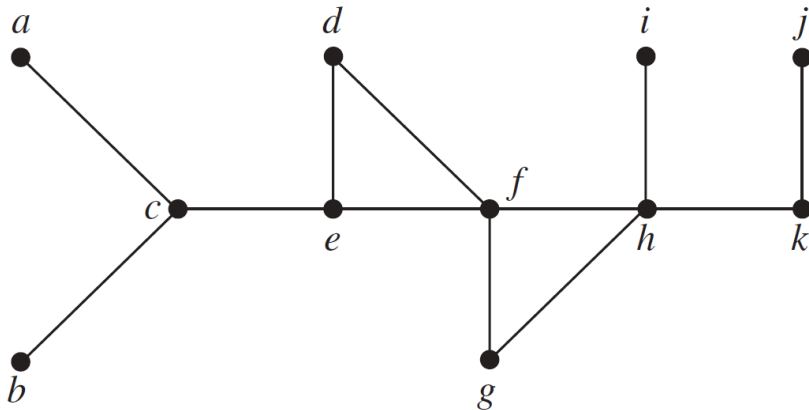
7 Spanning Trees



Example

7 Spanning Trees

Find a spanning tree of this graph using Breadth-First Search:



Breadth-First Search Algorithm

7 Spanning Trees

ALGORITHM 2 Breadth-First Search.

procedure *BFS*(G : connected graph with vertices $v_1, v_2, \dots, v_n$)

$T :=$ tree consisting only of vertex v_1

$L :=$ empty list

put v_1 in the list L of unprocessed vertices

while L is not empty

 remove the first vertex, v , from L

for each neighbor w of v

if w is not in L and not in T **then**

 add w to the end of the list L

 add w and edge $\{v, w\}$ to T



Table of Contents

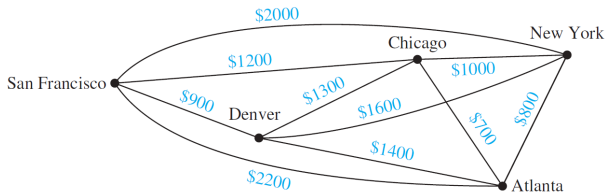
8 Min.Span.Trees

- ▶ Introduction to Trees
- ▶ Binary Search Tree
- ▶ Prefix code
- ▶ Decision tree
- ▶ Tree Traversal
- ▶ Infix, Prefix, and Postfix Notation
- ▶ Spanning Trees
- ▶ **Min.Span.Trees**
- ▶ Quiz

Introduction

8 Min.Span.Trees

A company plans to build a communications network connecting its five computer centers.



- Which links should be made to ensure that there is a path between any two computer centers so that the total cost of the network is minimized?
- We can solve this problem by finding a spanning tree so that the sum of the weights of the edges of the tree is minimized. Such a spanning tree is called a minimum spanning tree.



Definition

8 Min.Span.Trees

Definition 8.1

A **minimum spanning tree** in a connected weighted graph is a spanning tree that has the smallest possible sum of weights of its edges.



Prim's Algorithm

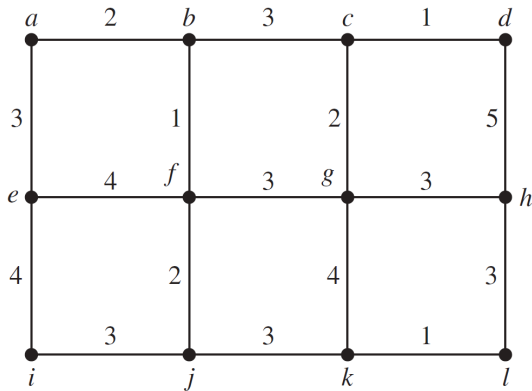
8 Min.Span.Trees

- Begin by choosing any edge with smallest weight, putting it into the spanning tree.
- Successively add to the tree edges of minimum weight that are incident to a vertex already in the tree, never forming a simple circuit with those edges already in the tree.
- Stop when $n - 1$ edges have been added.

Example

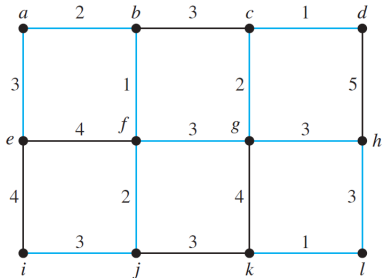
8 Min.Span.Trees

Use Prim's algorithm to find a minimum spanning tree in the graph



Solution

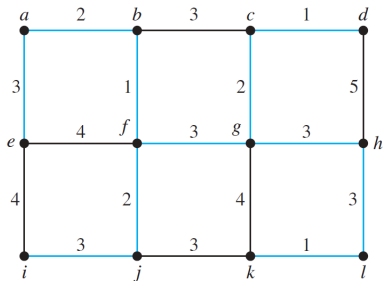
8 Min.Span.Trees



Choice	Edge	Weight
1	$\{b, f\}$	1
2	$\{a, b\}$	2
3	$\{f, j\}$	2
4	$\{a, e\}$	3
5	$\{i, j\}$	3

Solution

8 Min.Span.Trees



Choice	Edge	Weight
6	$\{f, g\}$	3
7	$\{c, g\}$	2
8	$\{c, d\}$	1
9	$\{g, h\}$	3
10	$\{h, l\}$	3

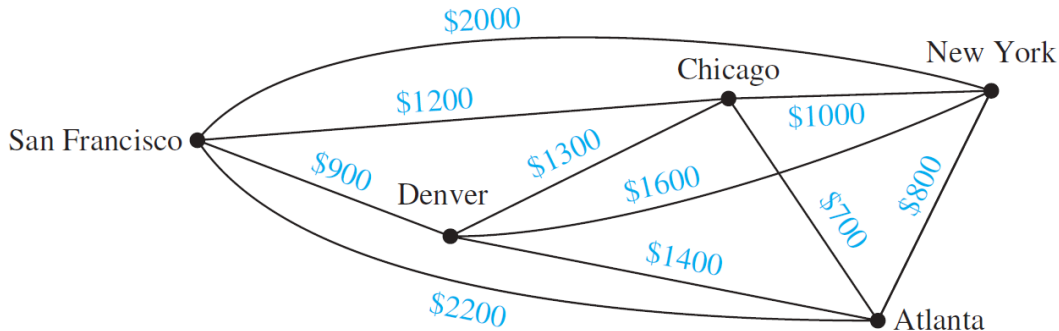


Choice	Edge	Weight
11	$\{k, l\}$	1

Example

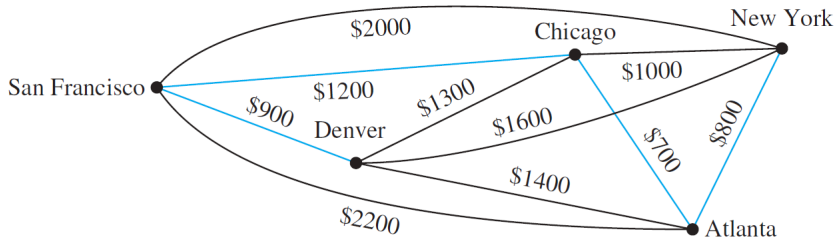
8 Min.Span.Trees

Use Prim's algorithm to find a minimum spanning tree in the graph



Solution

8 Min.Span.Trees



Choice	Edge	Cost
1	{ Chicago, Atlanta }	\$ 700
2	{ Atlanta, New York }	\$ 800
3	{ Chicago, San Francisco }	\$1200
4	{ San Francisco, Denver }	\$ 900
Total:		\$3600

Prim's Algorithm

8 Min.Span.Trees

ALGORITHM 1 Prim's Algorithm.

procedure *Prim*(G : weighted connected undirected graph with n vertices)

$T :=$ a minimum-weight edge

for $i := 1$ **to** $n - 2$

$e :=$ an edge of minimum weight incident to a vertex in T and not forming a
 simple circuit in T if added to T

$T := T$ with e added

return T { T is a minimum spanning tree of G }



Kruskal's Algorithm

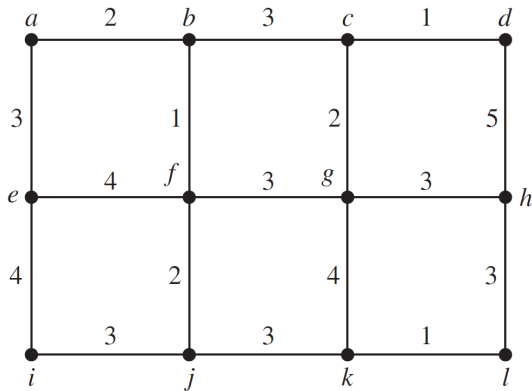
8 Min.Span.Trees

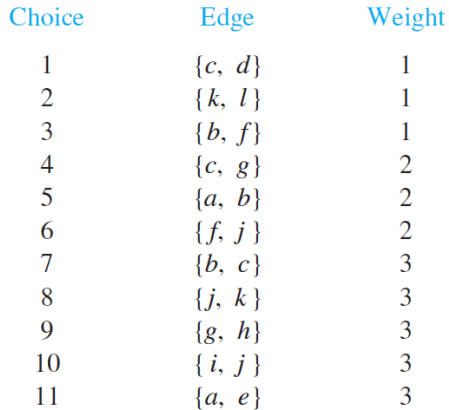
- Choose an edge in the graph with minimum weight,
- Successively add edges with minimum weight that do not form a simple circuit with those edges already chosen,
- Stop after $n - 1$ edges have been selected.

Example

8 Min.Span.Trees

Use Kruskal's algorithm to find a minimum spanning tree in the weighted graph





79 / 86

Kruskal's Algorithm

8 Min.Span.Trees

ALGORITHM 2 Kruskal's Algorithm.

```
procedure Kruskal( $G$ : weighted connected undirected graph with  $n$  vertices)  
 $T$  := empty graph  
for  $i$  := 1 to  $n - 1$   
     $e$  := any edge in  $G$  with smallest weight that does not form a simple circuit  
        when added to  $T$   
     $T$  :=  $T$  with  $e$  added  
return  $T$  {  $T$  is a minimum spanning tree of  $G$  }
```



Note

8 Min.Span.Trees

In Prim's algorithm, edges of minimum weight that are incident to a vertex already in the tree, and not forming a circuit, are chosen; whereas in Kruskal's algorithm edges of minimum weight that are not necessarily incident to a vertex already in the tree, and that do not form a circuit, are chosen.



Table of Contents

9 Quiz

- ▶ Introduction to Trees
- ▶ Binary Search Tree
- ▶ Prefix code
- ▶ Decision tree
- ▶ Tree Traversal
- ▶ Infix, Prefix, and Postfix Notation
- ▶ Spanning Trees
- ▶ Min.Span.Trees
- ▶ Quiz



Quiz

9 Quiz

Q1. Use **Huffman coding algorithm** to encode the word “**football**”. What is the **average number** of bits required to encode a character?

Select one:

- a. 2.35
- b. 2.5
- c. 2.45
- d. 2.25

Q2. Which codes are **prefix codes**?

- i. a: 1010, b: 010, c:1101, d:100
- ii. a:10, b:0101, c:1110, d: 1001

Select one:

- a. i
- b. ii
- c. None
- d. Both



Quiz

9 Quiz

Q3. Set up a **binary search tree** for the following list, in the given order, using alphabetical ordering: 5, 3, 6, 2, 4, 7.

Write the **preorder traversal** of the tree.

Select one:

- a. 5, 2, 3, 4, 6, 7
- b. 2, 4, 3, 7, 6, 5
- c. 5, 3, 2, 4, 6, 7
- d. 2, 3, 4, 5, 6, 7

Q4. How many edges does a full binary tree with 100 internal nodes have?

Select one:

- a. 200
- b. 100
- c. 101
- d. 201



Quiz

9 Quiz

Q5. Study the following prefix expression:

$$+ - * 235 / * 244$$

It will be evaluated to ...

Select one:

- a. 5
- b. 4
- c. None
- d. 6

Q6. Given the coding scheme: a:00, b:01, c:10, d:110, e:111. Find the word represented by: 11000010110111

Select one:

- a. abcde
- b. abcdee
- c. daabce
- d. dabbce

Ans: Q1: 2.5, Q2: i, Q3: 5, 3, 2, 4, 6, 7, Q4: 200, Q5: None, Q6: dabbce



Q&A

Thank you for listening!